LLM-Generated Binary Proofs: OSS-Verify on LPS and glibc on Picinae

Charles Averill Ilan Buzzetti Shreya Soman
Kevin Hamlen

The University of Texas at Dallas

Draft, October 25, 2026

Abstract

In October 2026, Anthropic and Theorem reported that language-model agents had written Rocq proofs about compiled RISC-V binaries of OSS-Fuzz targets. The proofs show that sanitizer builds of those targets never fault and behave like their plain builds [1, 2]. This note compares that work with our own experiment, in which Claude Opus 5.5, running in Claude Code, wrote Picinae proofs about functions lifted from the x86-64 glibc 2.35 shared library. The two efforts share a proof assistant and a binary-level starting point. They differ in almost everything else: the semantic model and how the program enters it, the kind of property proved, how specifications are obtained, the scale, and what has to be trusted. We also discuss the lifting workflow that a collaborator has written up as an agent skill, and how it relates to the lifter changes made in our experiment.

1 Introduction

The Anthropic post [1] argues that language models make formal verification of real software feasible at a much larger scale than before. Its headline result is 239 fuzz-test entry points in 123 open-source libraries, for which Claude proved that "there are no crashes on any input (up to 64 MB)". The proofs are stated over compiled RISC-V instructions under a formal model of a Linux process. The public repository from Theorem, `theorem-labs/oss-verify-proofs` [2], holds a checked release of part of that work.

Our project started from that post. We wanted to know how well the same kind of agent does on a different binary-verification platform, Picinae, with a different kind of specification: per-function functional correctness taken from the man pages, not crash-freedom of whole programs. The work so far is in `Examples/claude` of the Picinae repository, on the branch `llm_generated_proofs`.

Section 2 summarizes the two efforts. Section 3 compares the semantic models, Section 4 what the theorems say, Section 5 what makes our proofs hard, and Section 6 the methodology. Section 7 discusses lifting, including the collaborator's skill. Section 8 draws lessons.

	OSS-Verify (Anthropic / Theorem)	This project (Picinae)
Proof assistant	Rocq 9.2.0	Rocq 9.1
Target code	whole static RISC-V (RV64GC) executables: OSS-Fuzz harness, library, musl, sanitizer runtime; vector code paths avoided	individual functions of the x86-64 glibc 2.35 shared object, including the hand-written AVX2 assembly the machine actually runs, plus direct callees where needed
Program representation	ELF bytes loaded into a memory image, decoded by the Rocq ISA semantics	Picinae IL produced by the Ghidra p-code lifter `pcode2coq`
Semantics	LPS: RISC-V instructions plus Linux process semantics (system calls, start-up, budgets)	Picinae amd64 IL semantics; no OS model
Property	sanitizer build never errs and agrees with the plain build (also halting, bound and decidability shapes)	functional correctness against the man page, plus frame and register preservation
Correctness kind	safety over finite traces (agreement); total for the halts shape	partial correctness (holds if the function returns)
Specifications	generated mechanically from fuzz harnesses, from a fixed set of statement shapes	written by the model per function, from the man page and glibc behavior
Scale	post: 239 entry points, 123 libraries, about 1.8×10^8 lines of Rocq; repository release: 138 statements proved for 60 entry points in 25 libraries	78 proof files covering about 80 glibc entry points (among them the AVX2 `memmove`/`memcpy`, `strlen`, `strchr` and the short-string case of `strcmp`), about 1.9×10^4 lines of proofs and libraries
Agents	many autonomous agents with a shared workspace, over a few weeks	a single Claude Code session, with occasional human direction, over about three days
Unfinished proofs	not allowed: `Admitted` refused, axioms audited	four main theorems left `Admitted` because `Qed` is too slow, each with a note

Table 1: Summary. The OSS-Verify figures come from the post [1] and the repository README [2]; the two do not cover the same set of proofs (Section 6.4).

2 The two efforts at a glance

3 Semantic models

3.1 How the program enters the logic

In OSS-Verify the program is its bytes. The tool `elf-snapshot` turns an ELF file into a memory image (loadable segments, entry point, program headers). An OCaml plugin, `rocq-read-file`, reads the image into Rocq at compile time, and the RISC-V semantics in LPS decodes and executes instructions from that image. No disassembler or lifter is involved. The trusted path from binary to logic is the ELF loader and the image reader, and `Images.v` checks by computation that each image is well formed.

In Picinae the program is a Rocq function from addresses to IL statements. It is produced outside the logic by Ghidra (disassembly plus the SLEIGH specification of each instruction as p-code) and by `pcode2coq`, which translates p-code to Picinae IL. So the disassembler, the SLEIGH specifications and the translator are in the trusted base. The experiment underlined this risk: several translator bugs gave unsound lifts until we fixed them. For example, partial-register writes

clobbered the rest of the register, `LZCOUNT` was lifted as width minus popcount, and flag booleans had the wrong width (Section 7). A Picinae proof is a proof about the lifted IL; it is about the binary only to the extent that the lift is right.

Lifting does have advantages. The IL is architecture-neutral, so one symbolic interpreter and one simplifier (`psimpl`) serve every ISA Picinae supports. Proofs read in terms of IL variables and expressions, not instruction encodings. And because lifting works one function at a time, small proofs stay small.

3.2 Instruction semantics

LPS implements RV64GC from the RISC-V specification, as an executable semantics inside Rocq. The post adds that for the zlib case study Claude also wrote a formal x86 ISA semantics. [**TODO: Check whether that x86 semantics is published, and how its fidelity was checked.**] Picinae's amd64 semantics is SLEIGH's, as translated. Some instructions are modeled only approximately:

- the `LOCK` prefix lifts as a no-op, so atomicity is assumed, not modeled;
- bits 256–511 of the ZMM registers are not modeled; AVX/AVX2 code only ever zeroes them;
- SLEIGH user-defined operations that Ghidra leaves opaque (for example `vpcmpeqb_avx2` and `vpminub_avx2`) had to be spelled out byte by byte in the lifter.

3.3 Memory, faults and the environment

This is the most important semantic difference.

LPS models a Linux process. Memory is mapped by the loader and by `mmap`/`brk`, and an access outside it is an *error*. An instruction or system call that LPS does not interpret is also an error (`unmodeled`), and so are `abort()` (failed `assert`s and sanitizer reports) and illegal instructions (failed UBSan checks). The statements forbid every error, so a proof rules them all out. The environment is nondeterministic, in the way a sound model of Linux must be: `read` may return any nonempty prefix, `mmap` any free aligned address, and some system calls may fail spuriously. Proofs must cover every such choice. The model is single-threaded, runs no signal handlers, and has a simplified file system.

Picinae's memory is a total function from addresses to bytes. A load or store has a premise that the address is readable or writable (bits of the pseudo-variables `A_READ`/`A_WRITE`). When the premise fails, the access has no successor state; it is not an error. Picinae theorems are partial-correctness statements over the traces that exist, so a run that would fault simply has no continuation, and the theorem says nothing about it. As a result, our theorems do *not* establish memory safety. They say that *if* the function runs to its return, the result and the final memory are as specified. Picinae has no OS model; our functions make no system calls (the mutex functions reach the futex system call only inside a callee, which they assume through a contract).

Picinae's memory model also lets the vectorized string functions read past the end of a string without any special argument. For example, `__strlen_avx2` reads whole 32-byte and 128-byte blocks. In LPS the same reads must be shown to stay inside mapped pages, which is exactly the page-cross reasoning the glibc code does. Picinae proofs can skip that reasoning, but they do not get its benefit either.

3.4 Concurrency

Both models are single-threaded. For us this matters most for `pthread_mutex_lock` and `pthread_mutex_unlock`, which rank first and third among libc functions in our Firefox profile.

We prove their sequential behavior for every mutex kind, assuming `LOCK`-prefixed instructions are atomic and no other thread runs during the call. That is a real limitation for functions whose whole purpose is concurrency.

4 What gets proved

4.1 OSS-Verify statement shapes

Every OSS-Verify statement applies one of a few *shapes* (in `theories/Shapes.v`) to the memory images of a harness's builds. Each shape is proved, by `reflexivity`, equal to an LPS definition:

- **halts:** every execution of a build exits with an allowed status, under a memory-budget premise;
- **agrees:** on every input where the plain build stays within its budget, neither the plain build nor the sanitizer build errs, and the two have the same set of finite traces (stream bytes, exit status, final file contents);
- **disagrees at most b:** under a stated input distribution, the builds disagree with probability at most b;
- **agreement decidable:** some RISC-V program classifies exactly the agreeing inputs.

The published proofs are agreement statements. According to the README, no halts statement is proved in this release, so the proofs do not rule out non-termination. An agreement proof for an ASan build rules out every error the sanitizer detects on every covered input, which is what "no crashes" means in the post. It says nothing about functional correctness: a parser that returns a wrong result without faulting satisfies its statement.

4.2 Picinae glibc theorems

Our theorems are functional specifications of individual functions. Every file starts with a header that states the function's semantics, quotes the man page, names the model, and gives an English input/output specification. Typical main properties:

- exact results, for example `ffs` returns the 1-based index of the lowest set bit, and `rand_r` equals glibc's C reference;
- exact memory effects with a frame condition, for example memset sets `dest[0,n)` to `c` in every size class (including the `REP STOSB` path) and leaves every other byte unchanged;
- for non-leaf functions, correctness relative to a contract for each callee: `bsearch`, `lfind` and `lsearch` through a function-pointer comparator, `strtok_r`/`strsep` through PLT calls, `strdup` through an abstract heap for `malloc`, and the mutex functions through `__lll_lock_wait` and `__lll_lock_wake`.

Each function also gets generic theorems: the lifted program is well typed, code is not writable, callee-saved registers are preserved, and memory is preserved where the function does not write it.

Two theorems record places where glibc does not meet its man page. `sigisemptyset` tests only the low 32 bits of the set, so a set containing only real-time signals is reported empty. `a64l` does not sign-extend its 32-bit result, although the man page says it does. For each we proved the binary's behavior and a concrete counterexample to the man page, and confirmed the counterexample by running the system libc. [**TODO: Decide whether to report these upstream; check whether they are already known or documented.**]

4.3 Vacuity

A theorem can be true for an unintended reason. OSS-Verify guards against this in a few ways. Statements are generated from facts and rendered by a tool anyone can rerun. Each shape is shown equal to the LPS definition it stands for. `Images.v` checks that the images are well formed. Statements are fingerprinted, so a proof is always of the published statement.

Our main vacuity risk is in callee contracts. A contract that no code can satisfy makes any theorem that assumes it trivially true. This happened: an early `bsearch`/`lfind` contract omitted a conjunct of the callee's entry invariant (the return address on the stack), and a single-state trace refuted it. Every contract now has a `*_satisfiable` theorem that exhibits code meeting it. For some contracts the only witness we found is code that never returns, which is weak evidence.

5 What makes these proofs hard

OSS-Verify's achievement is breadth: hundreds of entry points, each a whole program, checked mechanically. Our proofs are few and small by comparison, but each one has to get through difficulties that the OSS-Verify statements do not ask for, or that its setup avoids. This section lists them, with the function where each one showed up. We also note where OSS-Verify's obligations are harder than ours.

5.1 Correct results, byte by byte

An agreement statement is satisfied by any program that neither faults nor behaves differently under a sanitizer. A `memcpy` that copies the wrong bytes satisfies it. Our `memcpy` theorem quantifies over every byte of memory: `dest[0,n)` must equal `src[0,n)` afterwards and every other byte must be unchanged. glibc meets this with stores that overlap on purpose:

- two possibly overlapping head and tail moves for small sizes;
- a 64-byte head, aligned 128-byte iterations and a saved tail for large sizes;
- REP MOVSB starting at a 64-byte boundary, followed by stores of a saved head.

Every path needs a coverage argument that the union of the stored offset ranges is exactly $[0, n)$. For `memmove`, the stores must also be shown not to clobber source bytes that have not yet been read. The OSS-Verify proofs do contain functional lemmas of this kind, for example a specification of musl's `memcpy`, because whole-program reasoning needs them. But they are internal lemmas about compiled C, not theorems about the code a production x86 system runs.

5.2 Hand-optimized code with many strategies

The functions Firefox calls most are glibc's hand-written AVX2 assembly, selected at load time by IFUNC. `__memmove_avx_unaligned_erms`, which serves both `memmove` and `memcpy`, has 351 instructions and chooses among at least seven strategies by size, overlap, 4 KiB aliasing and CPU-feature flags. One theorem over all of them was beyond what the proof engine could hold, so we split it into three theorems by size range: up to 256 bytes, the 128-byte loops, and REP MOVSB. Each theorem refutes the other paths' entries with `False` invariants. The non-temporal path's dispatch and entry are proven, but its interleaved multi-page store loops are admitted with a note; the overlapping path above 256 bytes is still open.

5.3 SIMD

The string functions compare 32 bytes at a time (VPCMPEQB), combine vectors with VPMINUB, collapse a comparison into a bit mask (VPMOVMSKB), and find the first set bit (TZCNT, which the lifter turns into a 33-iteration loop).

- **Lifting.** Ghidra leaves several AVX2 operations opaque. The lifter had to spell them out lane by lane, and to merge per-lane register writes whose naive translation doubled the term size with every lane.
- **Term size.** Stepping one VPMOVMSKB symbolically executes about 200 IL moves and produces a 32-way disjunction over the bytes of memory. The brute-force proof of a single mask fact took 106 s. We instead wrote instruction-level lemmas: a Coq function generates each lifted instruction, the lemma about it is proved once by induction over the lanes, and a proof steps the instruction with a single lemma application.
- **OSS-Verify avoids vector code.** Its statements include a variant for binaries that take vector code paths when the processor advertises the RISC-V vector extension: the process is told the machine has no vector unit, so the scalar paths are what gets verified.

5.4 Bit-level reasoning about positions

The correctness of the string functions rests on index arithmetic over masks. For `strlen`:

- bit j of the mask is set iff byte j of the block is zero;
- TZCNT gives the first such j, and with the bytes already known to be nonzero that determines the length;
- near a page end the mask of the aligned block is shifted right by s mod 32 with an *arithmetic* shift (SARX), and a separate argument shows that the sign-replicated high bits never produce the first set bit;
- the loop exit combines two 32-bit masks into one 64-bit value and uses a 64-bit TZCNT, which has to be split back into the two cases.

For `strchr`, a "hit" is a byte equal to 0 or to c; the loop detects it as a zero byte of $\min(x, x \oplus c)$. For `strcmp`, a stop is a byte that differs between the two strings or terminates the first, which the code computes as a zero lane of $\min(\mathrm{eq}(a, b), a)$.

5.5 Configuration in memory

The memcpy strategy depends on tunables that glibc sets at startup:

- the REP MOVSB threshold and its stop threshold;
- the non-temporal threshold;
- a CPU-feature word that disables REP MOVSB for short distances.

The proof reads them as symbolic memory, so they become preconditions and case splits. Some preconditions encode facts glibc guarantees elsewhere, for example that the REP MOVSB threshold is never below 256 bytes on AVX.

5.6 x86 semantics

Partial-register writes, the flags and their many encodings, REP prefixes (lifted as instructions that jump to themselves, so REP STOSB and REP MOVSB need their own invariants and the direction flag), and the LOCK prefix all had to be handled. Several needed lifter fixes for soundness

(Section 7). Branches on signed comparisons force the flags into the proof: `strcmp`'s page-safety test is a `JG` after a `CMP`, so the proof must recover the signed relation from the lifted `OF`, `SF` and `ZF` bit-expressions (the `OF` term turns out to be identically zero), and it cannot abstract those flags away the way the loop-only functions do.

5.7 The proof engine at the limit

A single glibc function can produce symbolic states larger than an OSS-Verify proof ever needs in one goal, because its paths rejoin and its values are wide. The resulting failures were mostly engineering, not mathematics:

- tactics that compare memory terms (`assumption`, `rewrite` with unification variables, `discriminate`) unfold them and run out of memory;
- `lia` runs out of memory on any hypothesis that mentions a load, and becomes very slow with several equations modulo 2^{64};
- the kernel check at `Qed` can take far longer than the tactics.

Each needed a work-around, recorded in the README: syntactic tactics, cleared contexts, explicit lemma instances, and extra cut points.

5.8 Where OSS-Verify's obligations are harder

The comparison cuts both ways. OSS-Verify proofs must:

- reason about whole programs, including start-up code, the C library and the sanitizer runtime;
- handle a nondeterministic operating system: short reads, arbitrary `mmap` addresses, spurious failures;
- show that every memory access stays inside mapped memory.

Our theorems are partial-correctness statements about single functions, and Picinae's memory model lets them ignore faults. The vectorized string functions read past the end of the string, and in Picinae that needs no argument. The same reads in LPS would need the page-boundary reasoning the glibc code is designed around.

6 Methodology

6.1 Where specifications come from

OSS-Verify turns testing infrastructure into theorems. An OSS-Fuzz harness already defines an entry point and an input format. The statement says that the harness behaves on all inputs the way fuzzing hopes it behaves on the inputs it tried. Producing a statement takes no understanding of the library; this is what makes 20,164 statements over 2,148 harnesses possible. It also limits what can be said, because there is no notion of the right output.

Our specifications are written by the model, function by function, from the man page and from reading the binary. They say much more, but they are a second artifact that must be reviewed against the documentation. For example, the headers note where glibc departs from the man page (`sigfillset` leaves out signals 32 and 33, which NPTL reserves), and where a specification needed a precondition the man page does not mention (`memcpy`'s loop theorem assumes `64 <= d`). We have not yet reviewed every specification against the C sources. [**TODO: Add a specification-review pass, ideally by someone other than the generating model.**]

6.2 Proof style

The OSS-Verify proofs work at whole-program scale. A typical proof file tree includes interaction-tree models of the program, simulation arguments between builds, and per-function lemmas for the C library. For example, the zstd `huf_decompress` proof contains a specification of musl's `memcpy` at its RISC-V entry address, with a non-overlap precondition much like ours. The proof scripts in the release are stored in a compressed replay encoding (`KReplay`) with comments removed, so they are checkable but not meant to be read. The post reports a typical proof of about 600,000 lines for binaries with a median of 18,000 instructions. The repository's `proofs/` directory holds about 3.9×10^7 lines of Rocq.

Picinae proofs follow the platform's Floyd–Hoare style. The prover places invariants at cut points (loop heads, joins, call sites). The symbolic interpreter (`x64_step`) and the simplifier (`psimpl`) step between them, and the remaining arithmetic and bit-vector goals are closed by hand. Proofs are written to be read, and the README records what we learned about keeping them fast. Much of the experiment's effort went into three performance problems:

- **Memory.** Conversion near symbolic memory terms made plain `reflexivity`, `assumption` and `rewrite` blow up past 8 GB. We replaced them with syntactic (`constr_eq`-guarded) versions and added cut points at every join.
- **Kernel time at `Qed`.** Several tactic proofs finish in seconds or minutes, but their `Qed` runs for more than 30 minutes. The cause we found is the kernel unfolding `N.ltb` against power-of-two literals. A `Strategy` declaration fixed some cases. Four theorems remain `Admitted` with a note (`pthread_mutex_lock`, `__memset_avx2_unaligned_erms`, `isinfl`, `rand_r`).
- **SIMD.** Stepping an AVX2 lane instruction symbolically builds a 32-lane term; `VPMOVMSKB` alone executes about 200 IL moves. Our response was a library of instruction-level lemmas (`amd64_avx2.v`). Each lemma describes a whole lifted instruction through a Coq generator convertible to the lifter's output, and is proved once by induction over the lanes.

The OSS-Verify release does not allow unfinished work: `Admitted` and new axioms are refused, and `Print Assumptions` is audited against an allowlist. We have no such audit yet; our libraries use `FunctionalExtensionality`, an axiom that is also on their allowlist. [**TODO: Add a `Print Assumptions` audit to the Makefile.**]

6.3 Agents and human involvement

The post describes many autonomous agents with little human involvement, sharing a workspace of finished proofs. In a 36-task benchmark, agents that shared proofs used 17% fewer tokens, and some succeeded after failing on their own. When an agent found a bug, it patched the code, regenerated the statement, and reproved it. Patches were logged and audited, and the reward was shaped to discourage changing the program to make a proof go through. Proofs reportedly cost up to hundreds of millions of tokens.

Our experiment used one agent session (Claude Code with Claude Opus 5.5) over about three days, with a human giving occasional direction. Examples: order the work from easiest to hardest; commit only when asked; leave a proof `Admitted` with a note when its tactics are fast but its `Qed` is slow; prioritize the functions that a profile of Firefox visiting a website shows libc spending the most time in. We did not patch glibc. Where the binary disagrees with its documentation, we prove what the binary does. [**TODO: Report token usage and wall-clock time per function.**]

6.4 Scale

The two OSS-Verify sources give different numbers. The post reports 239 entry points in 123 libraries. The repository release reports 138 proved statements, for 60 entry points in 25 libraries, out of 20,164 statements. It also explains that proofs were ported to the release's binaries and LPS version, which suggests the release is the subset ported so far. **[TODO: Confirm with the OSS-Verify authors.]** Either way, the gap with us is two to four orders of magnitude in proof volume. A single OSS-Verify proof is far larger than our whole project, mainly because each one reasons about a whole static executable, including the C library and the sanitizer runtime.

7 Lifting

7.1 Our pipeline

We lift from a Ghidra 11.4.2 project in which `libc.so.6` has been imported and analyzed. A headless driver, `lifter/pcode2coq_headless.py`, calls `pcode2coq`'s library entry point with these settings:

- relative addressing (entry point at 0) for single functions;
- global addressing for a group of functions lifted into one program (`f1+f2=name`), so that direct calls between them resolve;
- `strictFailure`, so unmodeled registers and operations abort the lift instead of becoming `Nop`;
- `typeCheck`, which appends a well-typedness theorem;
- static-memory hypotheses switched off;
- `canonicalTemps`, which numbers temporaries per instruction so that identical instructions lift to identical statements.

The AVX2 variants that the system actually runs are lifted by address. The lifter itself needed a series of soundness and capability fixes, kept on a branch of `pcode2coq`:

- partial-register writes merged into the base register;
- exact varnode widths and correct flag widths;
- `BOOL_NEGATE` as `xor 1`;
- `LZCOUNT` lifted correctly;
- intra-instruction branches, with the bit-scan loops of `BSF`/`TZCNT` bounded by `Rep`;
- AVX/AVX2 user operations spelled out;
- per-lane vector writes merged into one move;
- `LOCK` as a no-op.

[TODO: Merge these fixes upstream; several are soundness bugs that affect any amd64 lift.]

7.2 The collaborator's lifting skill

A collaborator working in parallel has written the lifting workflow as an agent skill (`SKILL.md`). It tells an agent to:

- run `pcode2coq_headless.py` from `Picinae/tools/ghidra-lifter` through Ghidra's `pyghidraRun` in headless mode, against a pre-existing Ghidra project;
- select functions with a Python regular expression (`-functionRegex`);
- always pass `-globalAddressing`, "so they can compose and be used together to reason about caller-callee relationships";
- pass `-typeCheck`;

- write the output to a subdirectory of `Examples` named after the binary, with one file per lifted function named after the function;
- use `-noanalysis` only when the binary has already been analyzed;
- leave `-prependProgName` and `-interleave` unset unless asked.

Compared with our pipeline:

- **The idea is the same.** Both drive `pcode2coq` headlessly through PyGhidra, against a pre-analyzed project, with type checking on. Writing the procedure down as a skill makes lifting something any agent can do reliably. Our session had to rediscover the procedure (Ghidra and JDK versions, PyGhidra installation) before lifting anything.
- **Addressing.** The skill always uses global addressing. We use relative addressing for leaf functions and global addressing only for groups that call each other. Global addressing composes better: caller and callee proofs can share addresses, which is what `perform_call`-style reasoning needs, and a function lifted alone can later join a group without being re-lifted. Relative addressing keeps leaf proofs independent of where the library is loaded. Given the skill's goal, global addressing everywhere seems the better default, and we should adopt it.
- **Selection and layout.** `-functionRegex` lifts many functions in one Ghidra start-up; our driver takes explicit names or addresses. The skill's layout (`Examples/<binary>/<function>.v`) differs from ours (`Examples/claude/<function>_amd64.v`). We should agree on one before merging.
- **Soundness options.** The skill does not mention `strictFailure`. Without it, the lifter emits `Nop` for unmodeled registers and operations, which silently changes the program; we recommend making it the default. The skill also predates the static-memory, canonical-temporaries and AVX changes; whether its script includes them depends on which `pcode2coq` it uses. [**TODO: Check the collaborator's `tools/ghidra-lifter` version against our `pcode2coq` branch.**]
- **Variant selection.** glibc picks string and memory routines at load time through IFUNC. A regular expression over symbol names can match the generic C version, which the system may never run. Our pipeline lifts the variant the machine selects (for example `__strlen_avx2`), found by address. The skill could say how to choose.

8 Discussion

Complementary properties. OSS-Verify proves a weak property (no detectable error, agreement with the plain build) about entire programs, mechanically, at large scale. We prove strong properties (functional correctness) about small units, with specifications that need judgment. The two connect naturally. Inside OSS-Verify proofs, agents already prove functional specifications of C library routines such as musl's `memcpy`, because whole-program reasoning needs them. A verified, reusable library of such specifications is what our project produces.

What Picinae could adopt.

- A checked release discipline: no `Admitted`, an axiom allowlist, and fingerprinted statements.
- Statements generated from existing artifacts. Test suites and fuzz harnesses could seed specifications that the model then strengthens.
- An explicit fault semantics, so that a theorem can also rule out faults instead of being silent about them.

What the Picinae approach offers.

- Functional specifications, readable against the documentation.
- An architecture-neutral IL: the same infrastructure already serves ARM, RISC-V and i386 in other Picinae examples.
- Proofs small enough for a human to read and maintain.
- Specification bugs found in a production C library (`sigisemptyset`, `a64l`) by comparing the binary with its man page. Crash-freedom statements cannot express such bugs.

Open questions.

- Can a model scale functional-correctness proofs the way OSS-Verify scaled agreement proofs, given that each specification needs review?
- How much of the lifter belongs in the trusted base? Validating lifted IL against an independent ISA semantics would close the gap with LPS's in-logic decoding.
- Our slowest proofs are limited by kernel checking time, not by the agent. What proof engineering would bring them to `Qed`?

References

[1] Anthropic. LLMs are making formal software verification feasible at a much larger scale. `https://www.anthropic.com/research/formal-verification`, October 2026.

[2] Theorem. OSS Verify: theorem statements about OSS-Fuzz fuzz targets as compiled RISC-V binaries, and machine-checked Rocq proofs of them. `https://github.com/theorem-labs/oss-verify-proofs`, 2026.

[3] Theorem. rocq-linux-process-semantics (LPS). `https://github.com/theorem-labs/rocq-linux-process-semantics`.